RAG Assignment Report: Baseline vs. Re-Ranking

Kathan Patel

19 October 2026

Setup

I compared a plain bi-encoder retriever against a bi-encoder plus cross-encoder re-ranker. On my test queries, re-ranking raised Hit@3 from 0.73 to 0.92 and added only about 11 ms per query. Both pipelines run over the same 156 chunks and use the same LLM prompt, so the only thing that changes is which 3 chunks the LLM sees.

- **Document:** I used the official FIDE Laws of Chess (FIDE Handbook E.I.01, 25 pages). It's dense with general rules, exceptions ("except when…"), penalties and prohibitions, which is exactly what makes retrieval hard.
- **Chunking:** I split each page into sentences on `.`, `;` and `:`, then packed them into chunks of at most 500 characters with a 100-character overlap. That gave 156 chunks, averaging 412 characters. I skipped the table of contents because it only repeats article titles and would act as a distractor.
- **Pipeline 1 (baseline):** `sentence-transformers/all-MiniLM-L6-v2` embeddings, normalised and stored in a FAISS `IndexFlatIP`, so the inner product equals cosine similarity. I retrieve the top 10 and pass the top 3 to the LLM to get Answer 1.
- **Pipeline 2 (re-ranking):** I take the same top 10, score all 10 (query, chunk) pairs with `cross-encoder/ms-marco-MiniLM-L-6-v2`, re-sort them, and pass the new top 3 to the LLM to get Answer 2.
- **LLM:** `Qwen/Qwen2.5-3B-Instruct`, downloaded from Hugging Face and run locally with greedy decoding, so the answers are deterministic. The prompt tells the model to answer only from the context passages.
- **Hardware:** a laptop with an NVIDIA RTX 4060 (8 GB). All three models ran on the GPU, and no external API was used.
- **Code:** everything is in a single Jupyter notebook, `RAG_Baseline_vs_ReRanking.ipynb`. GitHub repository: https://github.com/Kathan0/LMA-Assignment-1

To find failure cases, I wrote about 60 natural questions about the rules and labelled each with a short "gold phrase" from the clause that actually answers it. I then checked where that gold chunk ranked in each pipeline.

1 Architecture Comparison

The core difference is *when* the query and the passage meet. In a bi-encoder they never see each other inside the model. In a cross-encoder they are read together, token by token.

- **Bi-encoder (independent embeddings + cosine):** the query and each chunk go through the encoder separately. Each comes out as one fixed vector (384 dimensions for MiniLM), and relevance is just the cosine between the two vectors. Because chunk vectors don't depend on the query, I computed them once, offline (0.67 s for all 156 chunks). At query time I only embed the question and run a vector search.
- **Cross-encoder (joint self-attention):** the query and the chunk are concatenated into one input, `[CLS] query [SEP] chunk [SEP]`, and passed through the transformer together. Every query token can attend to every chunk token, and the model outputs a single relevance score rather than a vector. Nothing can be pre-computed, because the score only exists for that specific pair.

	Bi-encoder	**Cross-encoder**
Input	query and chunk encoded separately	query + chunk as one sequence
Output	one vector each, compared by cosine	one relevance score per pair
Pre-computation	chunk vectors indexed once	none possible
Cost per query	1 encoder pass + vector search	1 encoder pass **per candidate**
Strength	fast recall over the whole corpus	precise judgement of a few candidates

Table 1: Bi-encoder vs. cross-encoder.

In practice I saw this difference directly. The bi-encoder compresses a whole chunk into one point, so it rewards chunks that are about the same *topic* or share *words* with the query. The cross-encoder can notice that a specific clause actually answers the question. Examples are "the arbiter shall **refrain from** informing…" for a yes/no question about the arbiter, or "Players do **not** need to record the moves" in the rapid-play appendix. Single words like "not", "except" and "only" barely move a pooled embedding, but they change a cross-encoder's score a lot.

2 Qualitative & Rank Shift Analysis

In all 10 of my failure queries, re-ranking pulled the gold chunk into the top 3. In 7 of them it started outside the top 3, so the baseline LLM never saw the clause that answers the question.

2.1 Rank changes for the failure queries

What each failure type looked like in my document:

- **Keyword trap.** "Tie" never appears in the rulebook; the rule says "a player who draws his game scores a half point". The bi-encoder ranked the mobile-phone rule (12.3.b) higher, because it ends with "his score shall be a draw". Similarly, "forgot to press his clock" pulled in chess-clock chunks (Art. 6.7, 6.8, 6.12), while the answer sits in the arbiter's duties (13.6).
- **Exception.** For "write down my moves in a 30 minute game", the baseline retrieved the general recording rule (Art. 8.1, chunk 60). It missed that a 30-minute game is "rapidplay" (App. A.1), where "players do not need to record the moves" (A.2). The exception lives in the appendix, far from the general rule.
- **Negation / constraint.** The quickplay-appeal question depends on the phrase "a player **may not** appeal" (12.10). The baseline brought back the draw-claim chunks (10.2, 9.5), which are on topic but don't contain the restriction.
- **Buried at 4–10.** The rule that failing to promote counts as an illegal move (7.4.a) sat at rank 5, behind the rules on when a move or promotion counts as completed (Art. 4.6).

Failure type	Query	Gold clause (chunk)	Rank P1 → P2
Keyword trap	Can the arbiter tell me my opponent forgot to press his clock?	Art. 13.6 (93)	**#10 → #1**
Keyword trap	What score does a player receive for a tie?	Art. 11.1 (82)	**#7 → #2**
Exception	Do I need to write down my moves in a 30 minute game?	App. A.2 (95)	**#5 → #2**
Exception	Can I correct an illegal move in blitz after my opponent has already moved?	App. B.3.c (102)	**#5 → #2**
Exception	In a rapid game, what if the king and queen were placed on the wrong squares?	App. A.4.a (96)	#3 → #1
Negation	Can a player appeal the arbiter's ruling on a draw claim in the quickplay finish?	Art. 12.10 (89)	**#6 → #2**
Negation	Is the player who has the move allowed to go to the restroom?	Art. 12.2 (84)	#3 → #1
Negation	Can a draw offer be taken back?	Art. 9.1.b(1) (70)	#3 → #1
Buried 4–10	Is it illegal to leave a pawn on the last rank without replacing it?	Art. 7.4.a (54)	**#5 → #2**
Buried 4–10	Can I ask the arbiter how many moves have been played?	Art. 13.6 (92)	**#4 → #3**

Table 2: Gold-chunk rank before (P1) and after (P2) re-ranking. Bold rows: the gold chunk was outside the baseline's top 3 and entered it after re-ranking.

2.2 Answer 1 (baseline) vs. Answer 2 (re-ranked)

The clearest wins were cases where the baseline LLM gave a confident but wrong answer from nearby context:

- **Arbiter and the clock (13.6): hallucination.** Answer 1 said *"Yes, the arbiter can observe if your opponent has forgotten to press their clock…"*, which is the opposite of the rule. Answer 2 said *"No, the arbiter should refrain from informing a player… that they have not pressed their clock"*, which is correct.
- **Score for a tie (11.1): wrong rule.** Answer 1 borrowed the phone-penalty sentence and never gave a point value. Answer 2 correctly said a draw scores $\frac{1}{2}$ **point**.
- **30-minute game (A.2): misstated rule.** Answer 1 explained the standard-game recording rule (no writing moves in advance, and so on), which doesn't apply to rapid. Answer 2 correctly said recording isn't required in rapidplay, although it mixed in a condition about supervision from A.3.
- **Blitz illegal move (B.3.c): omitted clause.** Both answers said "No", but only Answer 2 included the key exception: the move can be corrected if both players agree.
- **Quickplay appeal (12.10): right answer, wrong reason.** Answer 1 said "No" only because *"the passage does not mention any process"*. Answer 2 grounded the "No" in the actual rule.

Re-ranking did not fix everything, and I think these cases matter just as much:

- **Already in the top 3:** for the restroom (12.2) and draw-offer (9.1) questions, the gold chunk was already at rank 3. Moving it to rank 1 produced practically identical answers. Ordering *inside* the top 3 made no difference to the LLM.
- **LLM still missed the clause:** in the rapid king/queen case, Answer 2 had the gold chunk but still left out "castling with this king is not allowed". In the pawn-promotion case, Answer 2

contradicted itself ("not illegal… however it must be promoted"). Better retrieval gives the 3B model the right evidence, but it doesn't guarantee the model reads it correctly.

2.3 Where re-ranking did not help

Case	Query	Gold clause	Rank P1 → P2
Recall ceiling	What should a player do if the piece he wants to promote to is not available?	Art. 6.12.b	#32 in corpus, never in top 10
Re-ranker demotes gold	Can a player castle when his king is currently in check?	Art. 3.8.b(2)	#5 → #8
Still outside top 3	What if I accidentally knock pieces off their squares?	Art. 7.3	#8 → #7

Table 3: Cases where re-ranking did not help.

- **Recall ceiling.** The re-ranker can only reorder what stage 1 gives it. The answer here (stop the clocks and call the arbiter) is phrased in clock language, so the bi-encoder ranked it 32nd. Both LLM answers then invented the same rule ("choose the next best available piece").
- **Demotion.** The MS MARCO cross-encoder preferred the touch-move castling rule (4.4), the rule against exposing your own king (1.2) and even a blitz illegal-move chunk over the dry list of conditions in 3.8.b. Both answers still said "No", but for invented reasons.
- **Partial help.** For knocked-over pieces, the exact rule (7.3) stayed at rank 7, but a closely related rule (7.5, "position before the irregularity shall be re-instated") moved into the top 3. Answer 2 was roughly right, while Answer 1 hallucinated that the pieces are "removed from the chessboard".

2.4 Aggregate retrieval metrics

I also scored both pipelines on 52 labelled questions. The set includes many easy questions, but because I wrote it while hunting for failures, it leans hard.

Pipeline	Hit@1	Hit@3	MRR@10
P1: bi-encoder only	0.462	0.731	0.611
P2: + cross-encoder	0.577	**0.923**	0.736

Table 4: Retrieval metrics on 52 labelled queries.

After re-ranking, the gold chunk's rank improved for 21 queries, stayed the same for 25 and got worse for 6. Hit@10 is 0.962 for both by construction, since re-ranking only reorders the same 10 candidates. Hit@3 matters most, because those are the only chunks the LLM sees, and it rose from 73% to 92%.

3 Latency Profiling

Re-ranking made retrieval about 2.7× slower: 17.6 ms instead of 6.4 ms. That extra 11 ms is only about 0.3% of the end-to-end time, because LLM generation takes over 3 seconds.

I measured every stage with `time.perf_counter()` after a few warm-up calls (the first GPU calls include CUDA start-up). Retrieval stages were timed over 200 runs (10 queries × 20 repetitions), and LLM generation over 10 runs. Everything ran on the RTX 4060 laptop GPU.

Stage	Mean (ms)	Median (ms)	p95 (ms)
P1 baseline retrieval (embed query + FAISS top 10)	6.42	6.36	6.85
Cross-encoder scoring of 10 pairs (alone)	11.20	10.95	14.31
P2 re-ranking retrieval (embed + FAISS + cross-encoder)	17.62	17.32	20.70
LLM generation (Qwen2.5-3B, same in both pipelines)	3,325	2,911	4,961
P1 end-to-end (retrieval + LLM)	3,332	2,918	4,967
P2 end-to-end (retrieval + re-rank + LLM)	3,343	2,929	4,978

Table 5: Latency per query on an RTX 4060 laptop GPU.

My observations:

- **The cross-encoder is the most expensive retrieval step.** Scoring just 10 pairs (11.2 ms) took longer than embedding the query and searching all 156 vectors (6.4 ms). That's because each pair is a full transformer pass over about 100–150 tokens, while the bi-encoder only encodes the short query once.
- **In a full RAG system the overhead is negligible.** Generation dominates, so the end-to-end difference (about 11 ms on about 3.3 s) is invisible to a user. Given that Hit@3 rose from 73% to 92%, I think the trade-off is clearly worth it.
- **The LLM time varies much more than retrieval does.** It depends on how long the answer is, which is why its p95 is far above its median. Retrieval times are very stable.

4 Scaling Question: Why Not Cross-Encode the Whole Collection?

A cross-encoder's cost grows linearly with the number of documents, *for every single query*. Nothing can be cached in advance. On my GPU that is about 1.08 ms per (query, chunk) pair, so a 1-million-chunk collection would take around 18 minutes to answer one question.

The reasons, as I see them:

- **No pre-computation.** A cross-encoder score only exists for a specific (query, chunk) pair. Unlike bi-encoder vectors, I can't compute anything ahead of time and store it in an index. Every new query means re-reading every chunk.
- N **full transformer passes per query.** For N chunks the cross-encoder runs N forward passes over the concatenated text. The bi-encoder runs one pass (the query) followed by a vector search, which approximate-nearest-neighbour indexes such as HNSW or IVF do in sub-linear time.
- **The bi-encoder moves its cost offline.** Embedding the corpus is paid once, when the index is built (0.67 s for my 156 chunks), and reused for every query.

I measured the cross-encoder on all 156 chunks (about 0.17 s per query) and extrapolated the per-pair cost:

This is why the two models are used together in a retrieve-then-re-rank design. The cheap bi-encoder handles **recall**: it narrows millions of chunks down to a small candidate set in milliseconds. The expensive cross-encoder handles **precision**: it carefully orders just those few. The cost of the second stage depends only on k (10 here), not on the size of the collection.

The trade-off is the recall ceiling I saw in Section 2.3. If the bi-encoder doesn't put the right chunk in the top k, the re-ranker can't rescue it. In a larger system I would increase k (for example to 50–100) or add a keyword retriever such as BM25 to improve stage-1 recall.

Collection size (chunks)	Cross-encoder over everything, per query	Bi-encoder top 10 + cross-encoder re-rank, per query
156 (this document)	0.17 s	∼18 ms
10,000	∼11 s	∼18 ms + ANN search
1,000,000	∼18 min	∼18 ms + ANN search
100,000,000	∼30 hours	∼18 ms + ANN search

Table 6: Measured (156 chunks) and extrapolated per-query cost.

Conclusion

On the FIDE Laws of Chess, adding a cross-encoder re-ranker on top of the bi-encoder was a clear improvement for very little cost. Hit@3 went from 0.73 to 0.92, and retrieval took about 11 ms longer.

- **Re-ranking fixes the failures I expected.** These were keyword traps, exceptions hidden in the appendices, and negated or role-specific rules. In those cases the baseline LLM hallucinated, misstated a rule or left out a clause, because the critical chunk was at ranks 4–10.
- **It isn't a cure-all.** It can't recover a chunk that stage 1 missed, it occasionally demotes the right chunk, and a small LLM can still misread good context.
- **The two-stage design is what makes it practical.** The bi-encoder gives cheap recall over the whole collection, and the cross-encoder gives precision on a handful of candidates. Running the cross-encoder over everything would scale linearly per query and quickly become unusable.

If I continued this work, I would raise stage-1 recall with a larger k or a hybrid BM25 + dense retriever, and test a stronger LLM. That would separate retrieval errors from generation errors more cleanly.